Lab Retele

Capitolele 2 & 3: DNS · HTTP · Flask · Docker · UDP · TCP · Scapy

01

DNS

Domain Name System: traducerea numelor in adrese IP

Cum ajunge 'fmi.unibuc.ro' la un IP

1. **Browser → Resolver local (8.8.8.8 / ISP)**

   Trimite query. Resolver-ul face munca in locul tau.

2. **Resolver → Root Server (.)**

   Root stie cine e responsabil de TLD-ul .ro → sec-dns-b.rotld.ro

3. **Resolver → TLD Server (.ro)**

   Serverul .ro stie ca ns.unibuc.ro e responsabil → 80.96.21.3

4. **Resolver → ns.unibuc.ro → raspuns final**

   Authoritative NS returneaza: fmi.unibuc.ro = 193.226.51.15 ✓

DNS = UDP port 53 | Iterativ: resolver face toti pasii | Cache TTL reduce nr. cereri | dig @8.8.8.8 fmi.unibuc.ro

Record types si cum le interogam

Tip	Ce stocheaza	Exemplu / Comanda
A	Nume -> IPv4	fmi.unibuc.ro -> 193.226.51.15
AAAA	Nume -> IPv6	google.com -> 2a00:1450::...
CNAME	Alias pentru alt nume	nlp.unibuc.ro -> nlp-unibuc.github.io
MX	Server email SMTP	dig @8.8.8.8 fmi.unibuc.ro MX
NS	Nameserver responsabil	dig @8.8.8.8 fmi.unibuc.ro NS
TXT	Date text arbitrare	dig @8.8.8.8 fmi.unibuc.ro TXT
PTR	IP -> Nume (reverse)	dig -x 193.226.51.15

Demo dig:

```
# Rezolutie iterativa manuala
dig @a.root-servers.net fmi.unibuc.ro
dig @sec-dns-b.rotld.ro fmi.unibuc.ro
dig @80.96.21.3 fmi.unibuc.ro

# Google DNS (face toti pasii)
dig @8.8.8.8 fmi.unibuc.ro

# Tipuri diferite
dig @8.8.8.8 fmi.unibuc.ro MX
dig @8.8.8.8 fmi.unibuc.ro NS
dig @8.8.8.8 fmi.unibuc.ro TXT
dig -x 193.226.51.15
```

In container: docker compose exec rt1 bash apoi dig @8.8.8.8 fmi.unibuc.ro

02

HTTP + Flask

Application Layer; server web in Python

Cum comunica clientul cu serverul

Metoda	Uz	Body
GET	Citire date / pagini	Nu
POST	Trimitere / creare resurse	Da
PUT	Actualizare completa	Da
DELETE	Stergere resursa	Nu

Status	Semnificatie
200 OK	Succes
201 Created	Resursa creata
400 Bad Request	Cerere invalida
404 Not Found	Nu exista
500 Server Error	Eroare server

Decoratoare, endpoint-uri, jsonify

```
from flask import Flask, request, jsonify
import socket

app = Flask(__name__)

# @app.route mapeaza URL -> functie Python
@app.route('/', methods=['GET'])
def index():
    return "Serverul functioneaza!"

# <item_id> = parametru dinamic in URL
@app.route('/item/<item_id>', methods=['GET'])
def get_item(item_id):
    return jsonify({"item_id": item_id})

# IP-ul real
@app.route('/ip', methods=['GET'])
def get_ip():
    ip = socket.gethostbyname(socket.gethostname())
    return jsonify({"ip": ip})

# POST: citeste body JSON
@app.route('/post', methods=['POST'])
def post_data():
    data = request.json  # dict Python din body
    value = data.get('value', 0)
    return jsonify({"received": value})

if __name__ == '__main__':
    # 0.0.0.0 = TOATE interfetele (nu doar localhost!)
    app.run(host='0.0.0.0', port=8001, debug=True)
```

- Importam Flask + request (date cerere) + jsonify (dict->JSON)
- @app.route: primul arg = URL, methods = metode acceptate
- <item_id> in ruta -> argument cu acelasi nume in functie
- gethostbyname(gethostname()) = IP containerului, NU 127.0.0.1
- request.json citeste body-ul cererii POST ca dict Python
- host='0.0.0.0' este OBLIGATORIU — altfel nu e accesibil din afara

Sintaxa esentiala pentru testarea API-urilor

```
# GET simplu
curl http://localhost:8001/

# GET cu parametru in URL
curl http://localhost:8001/item/10
# Output: {"item_id":“10"}

# GET IP-ul containerului daca serverul ruleaza in docker
curl http://localhost:8001/ip
# Output: {"ip":"172.111.0.x"}  ← NU 127.0.0.1!

# POST cu JSON body
curl -X POST http://localhost:8001/post \
     -H 'Content-Type: application/json' \
     -d '{"value": 10}'
```

Optiuni curl:

`-v`
Verbose — headere request + response

`-i`
Afiseaza headerele response

`-X METHOD`
Specifica metoda HTTP

`-H 'key: val'`
Adauga un header

`-d 'body'`
Body-ul cererii POST

`| python3 -m json.tool`
Formateaza JSON frumos

Gandeste pasii inainte sa scrii cod

T1 Serverul returneaza nume + nr. matricol

Ce returneaza un endpoint GET / daca ii spui return 'text'?
String simplu sau jsonify? Cum verifici cu curl?
Ce apare in terminal?

T2 {"item_id":"5"} cu curl

Cum definesti un parametru dinamic in ruta? (hint: <...> in path)
Cum ajunge valoarea ca argument in functia Python?
Ce cheie trebuie sa aiba dictionarul din jsonify?

T3 {"ip":"ip-containerului"}

Ruleaza si compara in container:
socket.gethostbyname('localhost') -> ?
socket.gethostbyname(socket.gethostname()) -> ?
Daca raspunsul e 127.0.0.1, ceva e gresit!

T5 Subnet mask pentru N noduri

Pasi: (1) cati biti de host trebuie pentru N+2 valori?
(2) prefix = 32 - biti_host
(3) cum construiesti masca dotted-decimal din prefix?
Verifica pe hartie: 250 noduri -> ? bits -> /? -> 255.255.255.?

Algoritmul teoretic in 3 pasi

1 noduri -> biti_host

hosturi_utile = 2^(biti_host) - 2 >= noduri
Deci: 2^(biti_host) >= noduri + 2

biti_host = ceil(log2(noduri + 2))

Ex: 250 noduri
2^x >= 252 -> x = 8

2 biti_host -> prefix

Total biti IP = 32

prefix = 32 - biti_host

Ex: biti_host = 8
prefix = 32 - 8 = 24
-> /24

3 prefix -> masca

24 de '1' urmate de 8 de '0':
11111111.11111111.11111111.00000000
= 255.255.255.0

Cum cu operatii pe biti?
(hint: 0xFFFFFFFF, <<, &)

noduri	biti_host	prefix	masca
250	8	/24	255.255.255.0
500	?	?	?
2	?	?	? (cazul minim)

03

Docker Build & Push

Pacheteaza serverul Flask si publica in Docker Hub

Reteta pentru imaginea ta

! Necesar cont pe Docker Hub

```
# Dockerfile

# Imaginea de baza - python:3.11-slim (~50MB)
FROM python:3.11-slim

# Directorul de lucru in container
WORKDIR /app

# Copiem requirements PRIMUL (cache Docker)
# Daca requirements.txt nu s-a schimbat,
# Docker nu reinstaleaza pip la fiecare build
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiem codul aplicatiei
COPY server.py .

# EXPOSE = documentatie, NU deschide portul!
# Port mapping se face la docker run -p
EXPOSE 8001

# Comanda la pornirea containerului
CMD ["python3", "server.py"]
```

requirements.txt:

```
flask
requests
beautifulsoup4
```

FROM: imaginea de baza. slim = mai mica, fara pachete extra

WORKDIR: toate COPY/RUN/CMD urmatoare sunt relative la asta

Copiem requirements INAINTE de cod — Docker cache-uieste layer-ul pip

EXPOSE: documentatie. Portul se deschide cu -p la docker run!

CMD: comanda la pornire. Oprire = Ctrl+C sau docker stop

Cei 5 pasi pentru a publica imaginea

1

```
docker build -t retele-server:latest .
```

-t = tag (nume:versiune) | . = contextul (directorul cu Dockerfile-ul)

2

```
docker run -p 8001:8001 retele-server:latest
```

Testeaza LOCAL inainte de push! -p host:container mapeaza portul

3

```
docker login
```

Autentificare la Docker Hub. O singura data pe masina.

4

```
docker tag retele-server:latest username/retele-server:latest
```

Adauga namespace-ul Docker Hub. Format: username/imagine:tag

5

```
docker push username/retele-server:latest
```

Incarca pe hub.docker.com/r/username/retele-server

Test final (simuleaza evaluatorul): docker pull username/retele-server:latest -> docker run -p 8001:8001 ... -> curl http://localhost:8001/

CHECKLIST

Ce trebuie sa functioneze la tema

T1

```
curl http://localhost:8001/
```

-> Returneaza numele si nr. matricol

T2

```
curl http://localhost:8001/item/5
```

-> Output EXACT: {"item_id":"5"} (nu 5.0, nu {"id":"5"})

T3

```
curl http://localhost:8001/ip
```

-> IP-ul real al containerului — NU 127.0.0.1!

T4

```
docker pull username/retele-server:latest  &&  docker run -p 8001:8001 ...
```

-> Imaginea functioneaza dupa pull din Docker Hub

T5

```
curl -X POST /subnet -H Content-Type:json -d '{"ip":"192.168.1.0","noduri":250}'
```

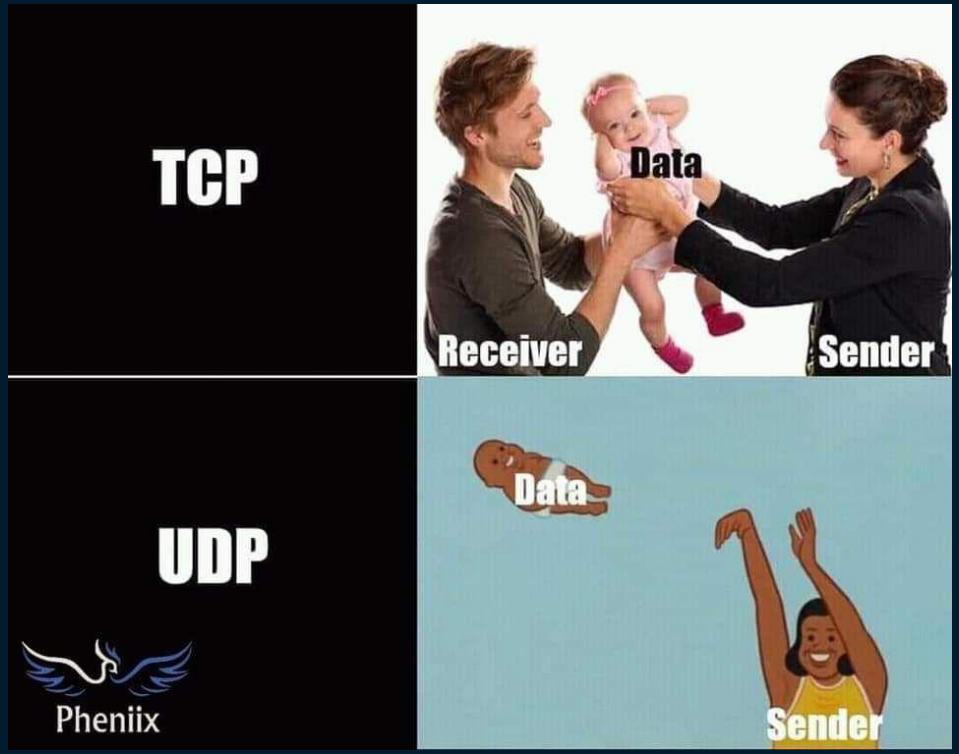
-> 255.255.255.0 sau /24

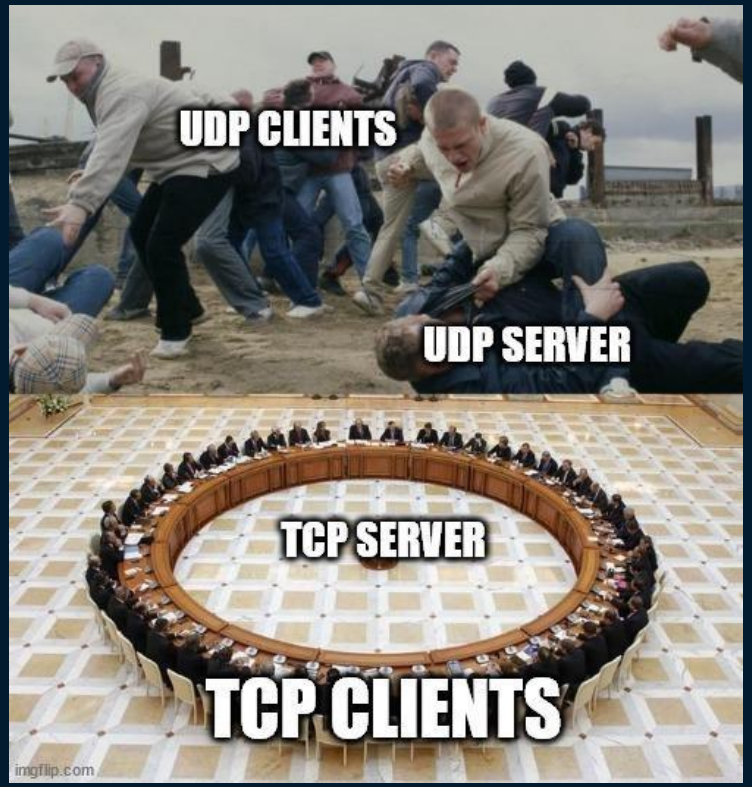
04

UDP and TCP

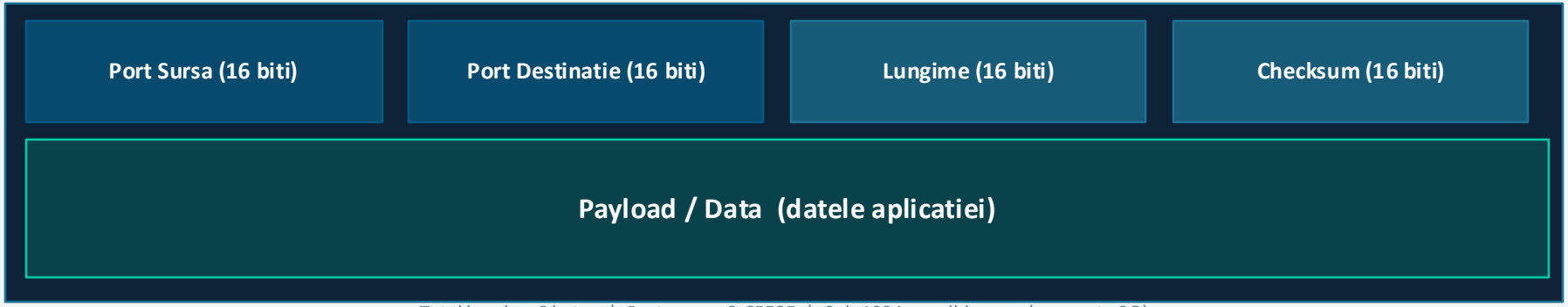
User Datagram Protocol: rapid, connectionless, fara garantii

Transmission Control Protocol: conexiune garantata, ordine, flux


TCP
Data
Receiver
Sender
UDP
Data
Sender
Pheniix


UDP CLIENTS
UDP SERVER
TCP SERVER
TCP CLIENTS
imgflip.com

UDP: 8 bytes, 4 campuri



Total header: 8 bytes | Port range: 0-65535 | Sub 1024 = well-known (rezervate SO)

Fara conexiune

Nu face handshake, trimite direct

Fara garantii

Pachete pot fi pierdute sau reordonate

Fara ordine

Nu garanteaza ordinea de sosire

Rapid + simplu

Overhead minim -> ideal real-time

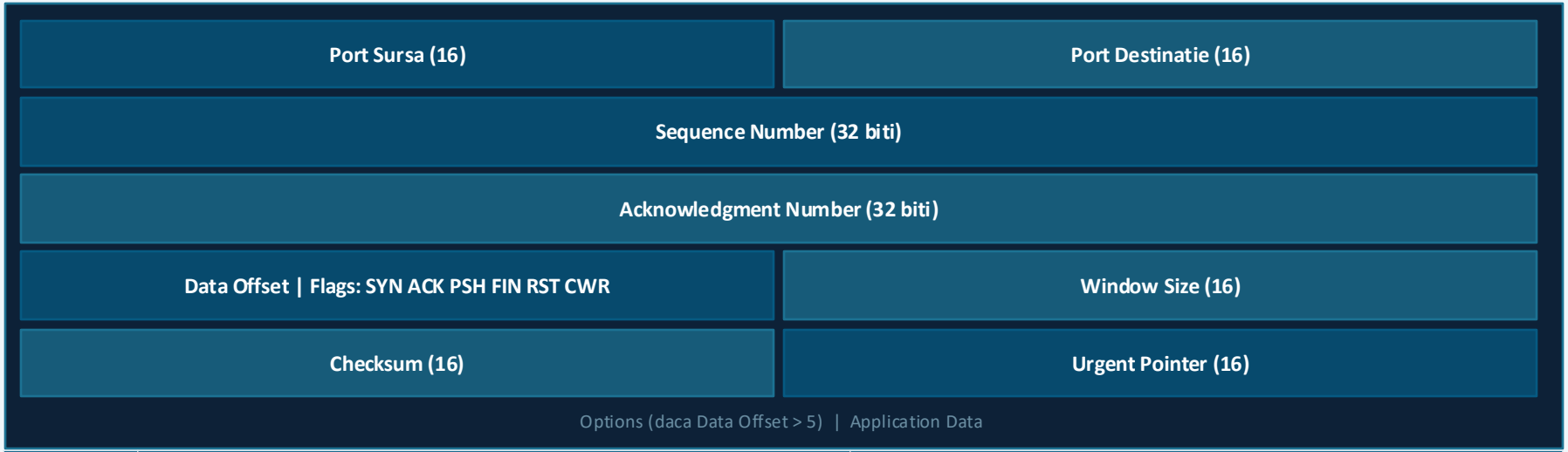
Uz: DNS VoIP gaming

Streaming, telemetrie IoT

Checksum optional

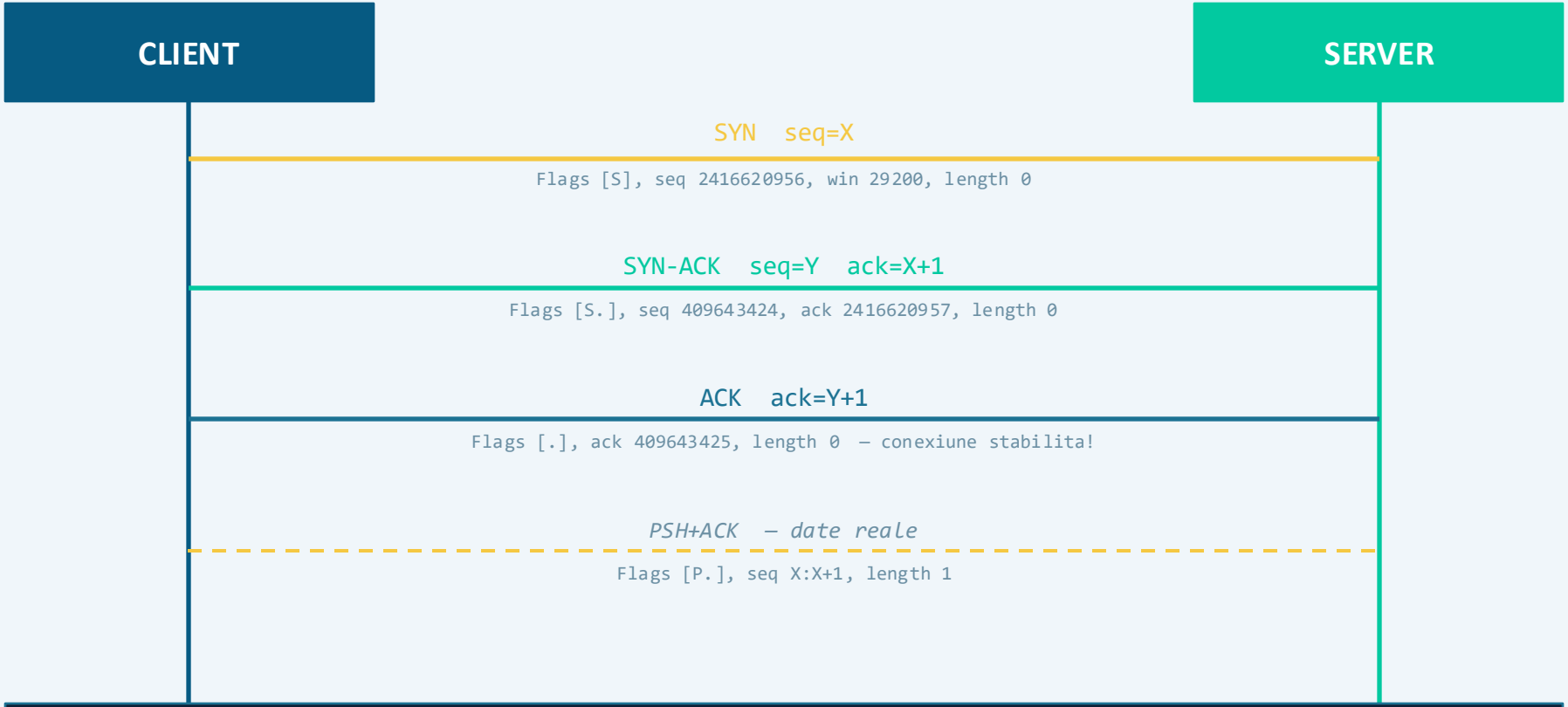
Verifica integritate daca activ

Structura unui segment TCP — 20+ bytes



Flag	Semnificatie	Cand apare
SYN	Synchronize — initiere conexiune	Handshake pasii 1 si 2
ACK	Acknowledge — confirmare primire	Aproape toate pachetele
PSH	Push — proceseaza imediat	Trimitere date reale
FIN	Finish — inchidere normala	Deconectare
RST	Reset — inchidere fortata	Erori, refuz conexiune

Stabilirea conexiunii inainte de date – 3-Way Handshake



Animatie simpla 3-way handshake

Socket API

Server:

```python
import socket

# AF_INET=IPv4, SOCK_DGRAM=UDP
sock = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM,
    proto=socket.IPPROTO_UDP)

# 0.0.0.0 = TOATE interfetele (nu localhost)
sock.bind(('0.0.0.0', 10000))
print('Server UDP pornit...')

while True:
    # recvfrom = BLOCANT pana sosesc date
    # returneaza (bytes, (ip_sursa, port_sursa))
    data, address = sock.recvfrom(1024)
    print(f'De la {address}: {data}')

    # Trimite raspuns la aceeasi adresa
    sock.sendto(b'ACK: ' + data, address)
```

Client:

```python
import socket, sys

sock = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM,
    proto=socket.IPPROTO_UDP)

mesaj = sys.argv[1].encode('utf-8')

# Nu e nevoie de bind() sau connect()
# sendto aloca automat un port efemer
sock.sendto(mesaj, ('rt1', 10000))

# Asteapta raspuns (blocant)
data, addr = sock.recvfrom(1024)
print(f'Raspuns: {data}')

sock.close()
```

SOCK_STREAM: stream de bytes ordonat si garantat

Server:

```
import socket

sock = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM,  # TCP!
    proto=socket.IPPROTO_TCP)

# Refoloseste portul dupa restart
sock.setsockopt(socket.SOL_SOCKET,
                socket.SO_REUSEADDR, 1)
sock.bind(('0.0.0.0', 10000))
sock.listen(5)  # max 5 conexiuni pending

while True:
    # accept() = BLOCANT, executa handshake
    # Returneaza socket NOU per client!
    conn, addr = sock.accept()
    print(f'Nou: {addr}')

    data = conn.recv(1024)
    conn.send(b'Hello TCP!')
    conn.close()  # inchide CONEXIUNEA
    # sock ramane deschis pt noi clienti
```

Client:

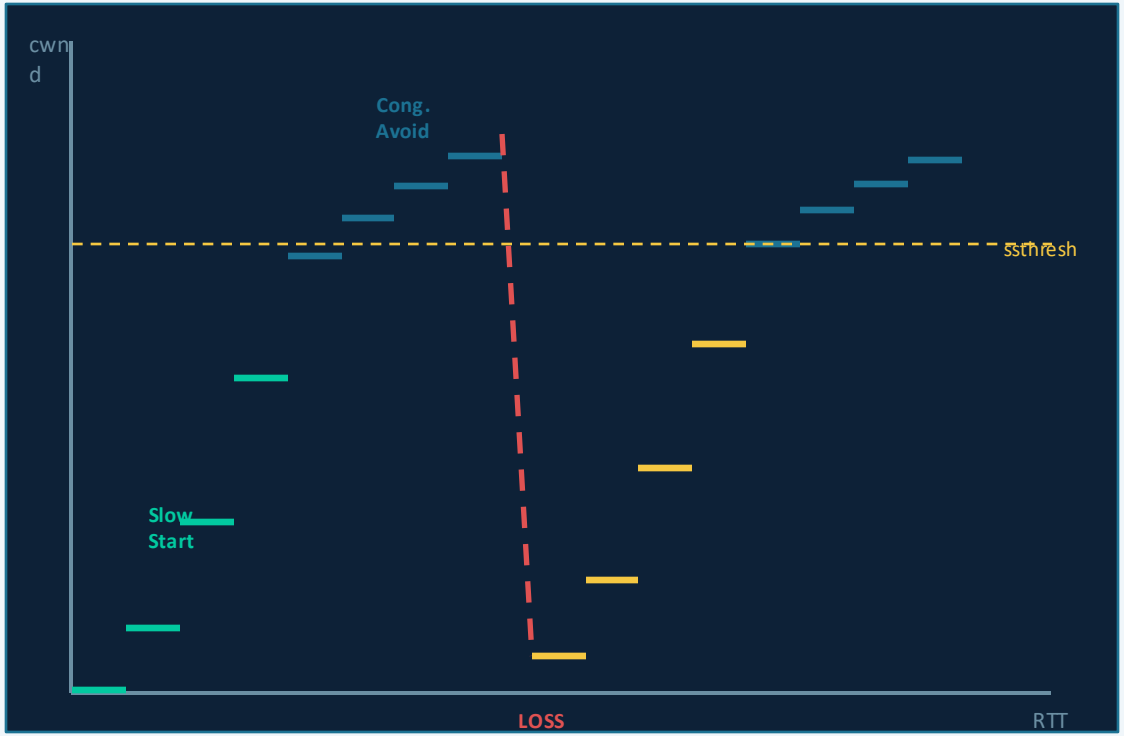
```
import socket, sys

sock = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM,
    proto=socket.IPPROTO_TCP)

# connect() executa 3-way handshake
# Blocant pana conexiunea e stabilita
sock.connect(('rt1', 10000))

mesaj = sys.argv[1].encode('utf-8')
sock.send(mesaj)

data = sock.recv(1024)
print(f'Raspuns: {data}')

sock.close()
```

06

TCP Congestion Control

Cum evita TCP sa blocheze reteaua: cwnd, ssthresh, slow start

Fereastra de congestionare (cwnd):



Slow Start

cwnd creste EXPONENTIAL (+1 MSS = maximum segment size per segment confirmat). Rapid la inceput. Se opreste la ssthresh sau pierdere.

Congestion Avoidance

Dupa ssthresh, cwnd creste LINEAR (+1 MSS/RTT). Crestere additiva, prudenta.

Pierdere (loss)

3 ACK duplicate = Fast Retransmit. ssthresh = cwnd/2. cwnd = 1 MSS. Scadere multiplicativa.

Fast Recovery

Dupa retransmisie, trece direct la Congestion Avoidance de la noul ssthresh. Nu reia Slow Start.

Animatie care explica pe scurt

07

Scapy Fundamentals

Construieste, trimite si captureaza pachete din cod Python

Operatorul ‘/’ - stivuieste straturi

Ce face Scapy

- Captureaza pachete live (sniff)
- Construieste pachete camp cu camp
- Trimite pe retea (send, sr1)
- Citeste/scrie fisiere .pcap
- Dezasambla bytes bruti din socket

```
Socket obisnuit: tu dai datele, SO construieste
headerele
Scapy: tu construiesti FIECARE CAMP manual
Campurile None = calculate automat la trimitere
```

Instalare:

```
pip install --pre scapy[complete]
```

Necesita root: docker compose exec --user root rt1 scapy

```
from scapy.all import *

# Operatorul / stivuieste straturi
pachet = Ether() / IP(dst='198.10.0.2') \
         / UDP(dport=10000) \
         / Raw(b'salut')

pachet.show()
###[ Ethernet ]###
  dst= 02:42:c6:0a:00:02  # ARP automat
  src= 02:42:c6:0a:00:01
###[ IP ]###
  chksum= None  # calculat la trimitere
  src= 198.10.0.1  # din interfata locala
  dst= 198.10.0.2
###[ UDP ]###
  len= None   # calculat la trimitere
  dport= 10000
###[ Raw ]###
  load= 'salut'
```

IP, UDP, TCP, ICMP, DNS

```
# ─── IP ──────────────────────────────────────────────
ip = IP(dst='8.8.8.8', ttl=64)
ip.src = '1.2.3.4'   # spoofing (posibil cu Scapy!)

# ─── UDP ─────────────────────────────────────────────
udp = UDP(sport=4444, dport=53)
# len si chksum = calculate automat

# ─── TCP ─────────────────────────────────────────────
tcp = TCP(sport=12345, dport=80)
tcp.flags  = 'S'    # SYN
tcp.flags  = 'SA'   # SYN-ACK
tcp.flags  = 'PA'   # PSH + ACK (date reale)
tcp.flags  = 'FA'   # FIN + ACK
tcp.seq    = 1000
tcp.ack    = 0
tcp.window = 8192
print(TCP.flags.names)  # 'FSRPAUECN'

# ─── ICMP ────────────────────────────────────────────
icmp = ICMP(type=8)  # 8=Echo Request, 0=Echo Reply

# ─── DNS ─────────────────────────────────────────────
dns = DNS(rd=1, qd=DNSQR(qname='fmi.unibuc.ro'))
# rd=1 = recursion desired

# ─── Combini si trimiti ──────────────────────────────
pachet = IP(dst='198.10.0.2') / UDP(dport=10000) / Raw(b'test')
send(pachet)  # Layer 3 — Ethernet completat de SO
```

Captureaza si proceseaza pachete

sniff():

```
# Captare simpla — Ctrl+C pt stop
pachete = sniff()
print(pachete)
# <Sniffed: TCP:2 UDP:1 ICMP:3>

# Fix N pachete
pachete = sniff(count=10)

# Filtru BPF — aceeasi sintaxa ca tcpdump
pachete = sniff(filter='icmp', count=5)
pachete = sniff(filter='udp port 10000')
pachete = sniff(filter='tcp and host 8.8.8.8')

# Handler in timp real
def handler(p):
    if p.haslayer(IP):
        print(f'{p[IP].src} -> {p[IP].dst}')
    if p.haslayer(Raw):
        print(f'  Payload: {p[Raw].load}')

# store=False = nu tine in RAM
sniff(prn=handler, filter='udp', store=False)
```

Navigare + pcap:

```
# Verifica prezenta unui layer
if pachet.haslayer(TCP):
    print('E TCP')

# Acceseaza campuri direct
src  = pachet[IP].src
port = pachet[TCP].dport
ttl  = pachet[IP].ttl
load = pachet[Raw].load

# Sumar compact (o linie per pachet)
pachet.summary()
# 'Ether/IP/UDP/DNS Qry"fmi.unibuc.ro."'

# Hex + ASCII (ca tcpdump -XX)
hexdump(pachet)

# ─── .pcap read/write ─────────────────
# Scrie
pachete = sniff(count=20)
wrpcap('captura.pcap', pachete)

# Citeste (interoperabil cu tcpdump/Wireshark)
pachete = rdpcap('captura.pcap')
for p in pachete:
    if p.haslayer(ARP):
        print(p[ARP].psrc)
```

08

Scapy Send Functions

send, sr, sr1, sendp; trimiterea pachetelor la Layer 2 si 3

Layer 3 (IP) si Layer 2 (Ethernet): cand sa folosesti ce

Functie	Layer	Ce face
send(p)	3 — IP	Trimite, nu asteapta raspuns. Ethernet completat de SO.
sr(p)	3 — IP	Send+Receive toate raspunsurile. Returneaza (answered, unanswered).
sr1(p)	3 — IP	Send si returneaza PRIMUL raspuns. Cel mai folosit.
sendp(p)	2 — Ethernet	Control total inclusiv MAC-uri. Util pt ARP, VLAN etc.
srp(p)	2 — Ethernet	Send+Receive la Layer 2.
srp1(p)	2 — Ethernet	Send si primul raspuns la Layer 2.

```
# send — trimite, nu asteapta
send(IP(dst='198.10.0.2') / UDP(dport=10000) /
Raw(b'salut'))

# sr1 — trimite si returneaza primul raspuns (timeout in
sec)
raspuns = sr1(IP(dst='8.8.8.8') / ICMP(), timeout=2,
verbose=0)
if raspuns:
    print(f'Reply de la {raspuns[IP].src},
TTL={raspuns[IP].ttl}')

# DNS query cu sr1
dns_q = IP(dst='8.8.8.8') / UDP(dport=53) / DNS(rd=1,
qd=DNSQR(qname='fmi.unibuc.ro'))
r = sr1(dns_q, timeout=3, verbose=0)
if r and r.haslayer(DNS):
    print(r[DNS].an.rdata)   # -> 193.226.51.15
```

```
# sr — scan hosturi UP (trimite la tot
subnet)
answered, unanswered = sr(
    IP(dst='198.10.0.0/24') / ICMP(),
timeout=2, verbose=0)
print(f'Hosturi UP: {len(answered)}')
for sent, received in answered:
    print(f'  {received[IP].src}')
```

sniff + ping simultan

Terminal 1 -> rt1 root -> asculta ICMP:

```
from scapy.all import *

def show_icmp(p):
    if p.haslayer(ICMP) and p.haslayer(IP):
        tip = 'REQUEST' if p[ICMP].type == 8 else 'REPLY  '
        print(f'ICMP {tip}: {p[IP].src} -> {p[IP].dst}  TTL={p[IP].ttl}')

sniff(prn=show_icmp, filter='icmp', store=False)
```

Terminal 2:

```
docker compose exec rt2 bash
ping -c 4 rt1
```

Output asteptat:

```
ICMP REQUEST: 172.111.0.3->172.111.0.2 TTL=64
ICMP REPLY  : 172.111.0.2->172.111.0.3 TTL=64
(repetat de 4 ori)
```

Harta capitolelor 2 & 3:

DNS
rezolutie, dig, record types

HTTP
GET/POST, status codes

Flask
@route, jsonify, request.json

Docker
build, tag, push, registry

UDP
SOCK_DGRAM, sendto/recvfrom

TCP
3-way hs, flags, seq/ack

Congestion
cwnd, ssthresh, slow start

Scapy
/ operator, sniff, sr1, haslayer